



ROI Training

Anthropic Models on Amazon Bedrock

Module 6:

Practical Architecture and
Deployment Patterns

Course Contents

Module 1: Foundations: Generative AI and Claude Models on AWS

Module 2: Claude API Fundamentals and Prompt Engineering

Module 3: Data Preparation and Knowledge Bases for RAG

Module 4: Tool Use and Agentic Workflows with Claude

Module 5: Security, Governance, and Guardrails

► **Module 6: Practical Architecture and Deployment Patterns**

Module 7: Claude Code: Building Bedrock Applications

Module 8: Amazon Kiro and Enterprise Code Generation



Module Objectives



End-to-End Data Flows

Ingestion, retrieval, invocation, monitoring



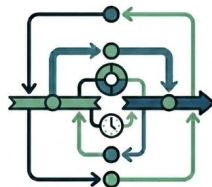
Integration Patterns

Lambda, ECS/EKS, and API Gateway



Optimize Costs

Model selection, service tier strategies, and cost optimization



Implement Resilience

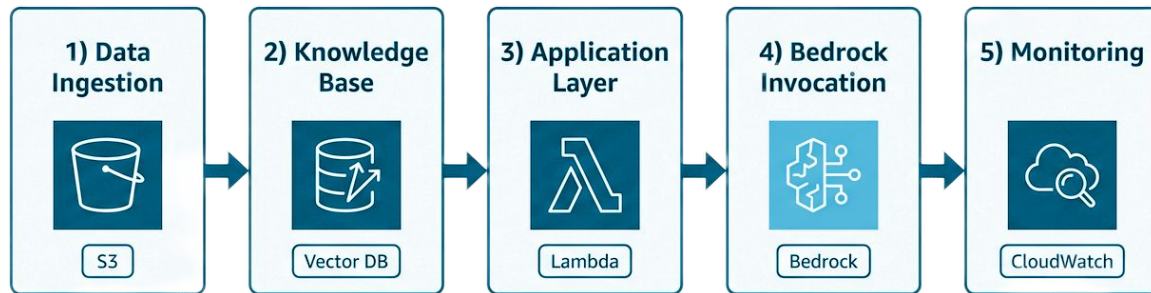
Build robust systems with retry logic, circuit breakers, and failover strategies



ROI Training

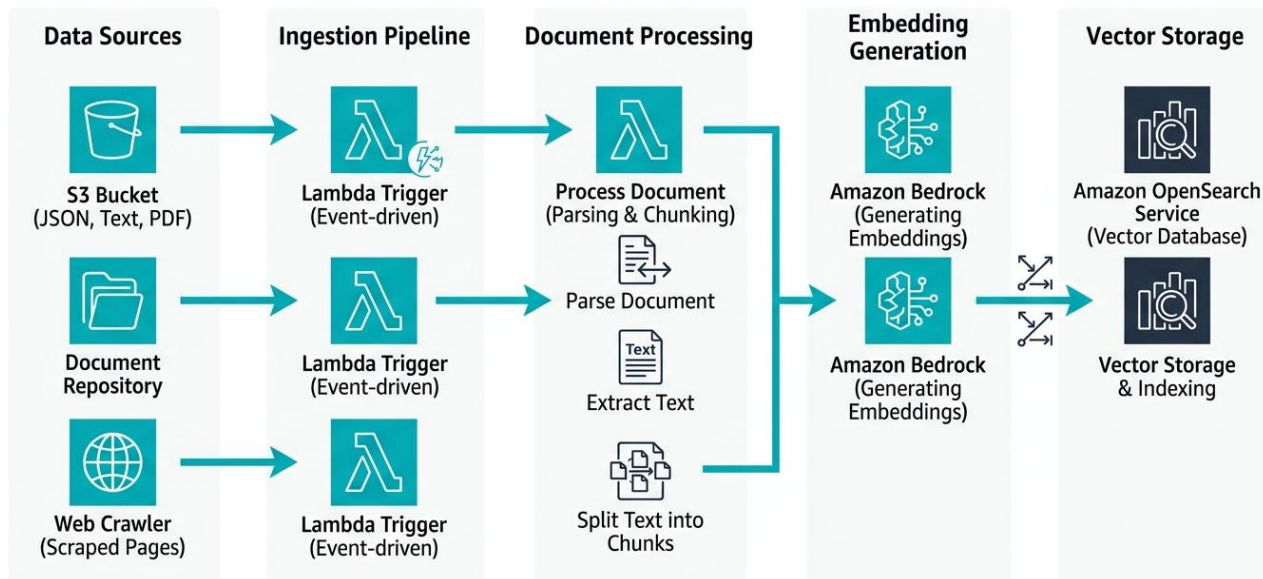
Data Flow

The Five-Stage Pipeline

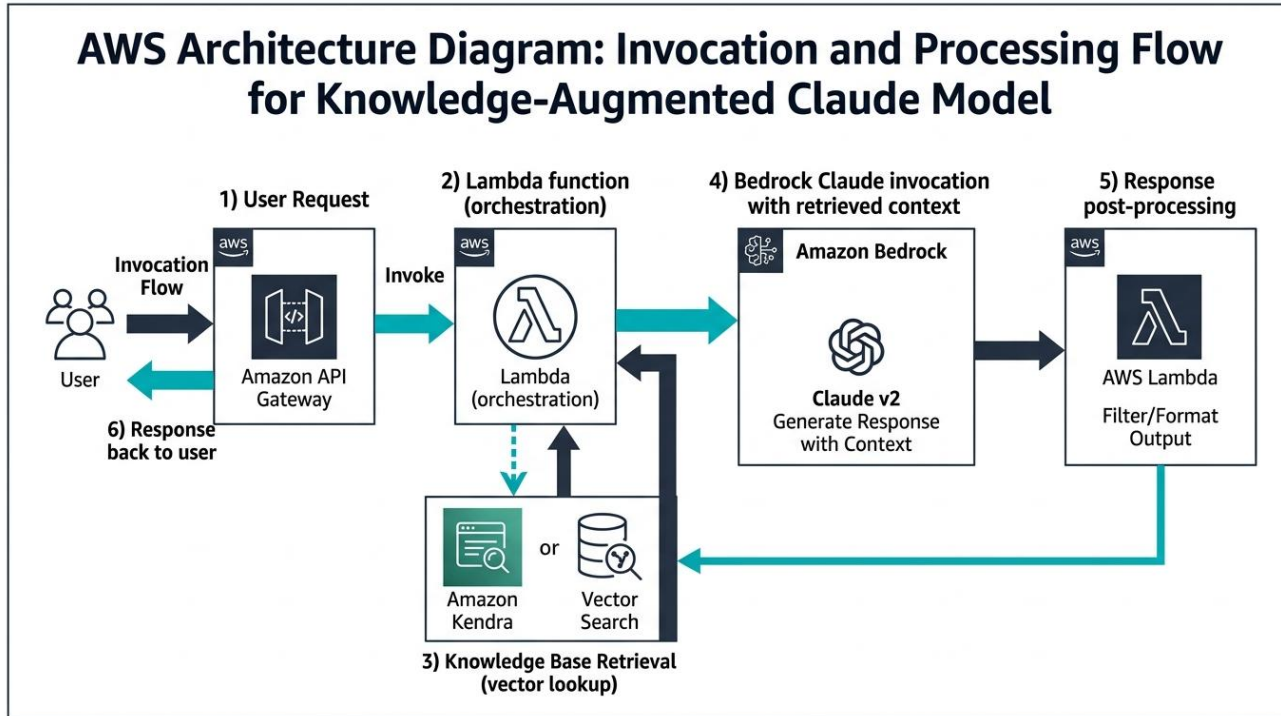


Data Flow: Ingestion and Retrieval

AWS DATA INGESTION FLOW: RAW DATA TO VECTOR STORE



Data Flow: Invocation and Processing



Synchronous vs. Asynchronous Patterns

Pattern	Use Case	Pros	Cons
Synchronous	Real-time chat, interactive APIs	Immediate response	Timeout risk, scaling limits
Asynchronous	Batch processing, complex analysis	Reliable, scalable	User waits for results
Hybrid	Long-running with progress	Best of both	More complexity

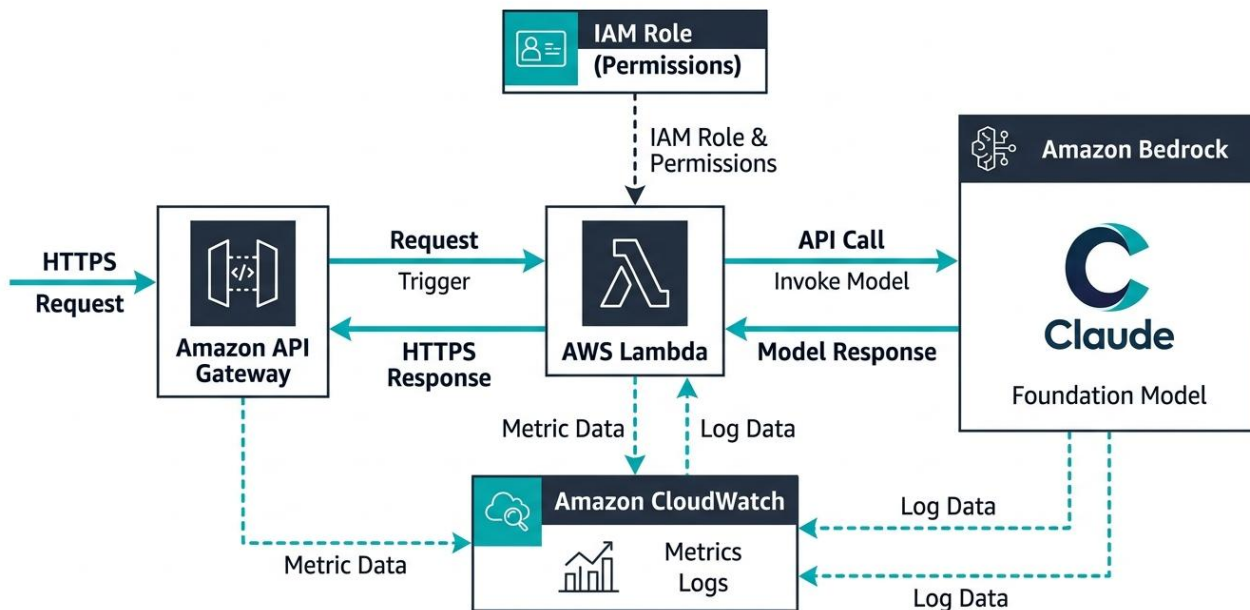


ROI Training

Integration

Lambda Integration Pattern

AWS Lambda Integration with Amazon Bedrock Architecture



AWS Lambda Best Practices for Amazon Bedrock

1

Use provisioned concurrency for low-latency requirements

2

Configure appropriate timeout (start with 60 seconds)

3

512MB-1024MB memory is usually sufficient

4

Enable X-Ray tracing for latency analysis

5

Reuse Bedrock client across invocations

Lambda Function Example

```
import boto3, json

bedrock = boto3.client('bedrock-runtime')

def handler(event, context):
    body = json.dumps({
        'anthropic_version': 'bedrock-2023-05-31',
        'messages': [{ 'role': 'user',
                        'content': event['prompt'] }],
        'max_tokens': 1024
    })
    resp = bedrock.invoke_model(
        modelId='anthropic.claude-sonnet-4-6', body=body)
    result = json.loads(resp['body'].read())
    return {'body': result['content'][0]['text']}
```

ECS/EKS for Containerized Workloads



Amazon ECS

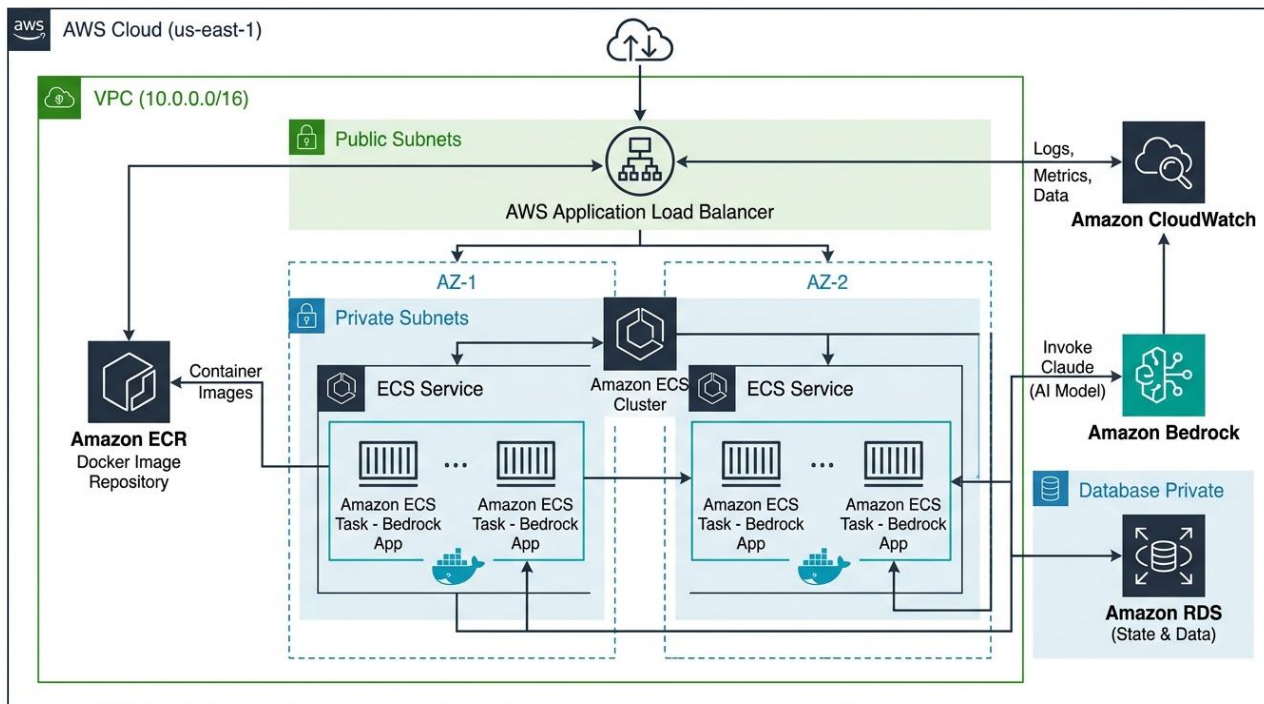
- Long-running stateful processes
- Predictable high-volume traffic
- Existing container infrastructure
- Simpler AWS-native orchestration



Amazon EKS

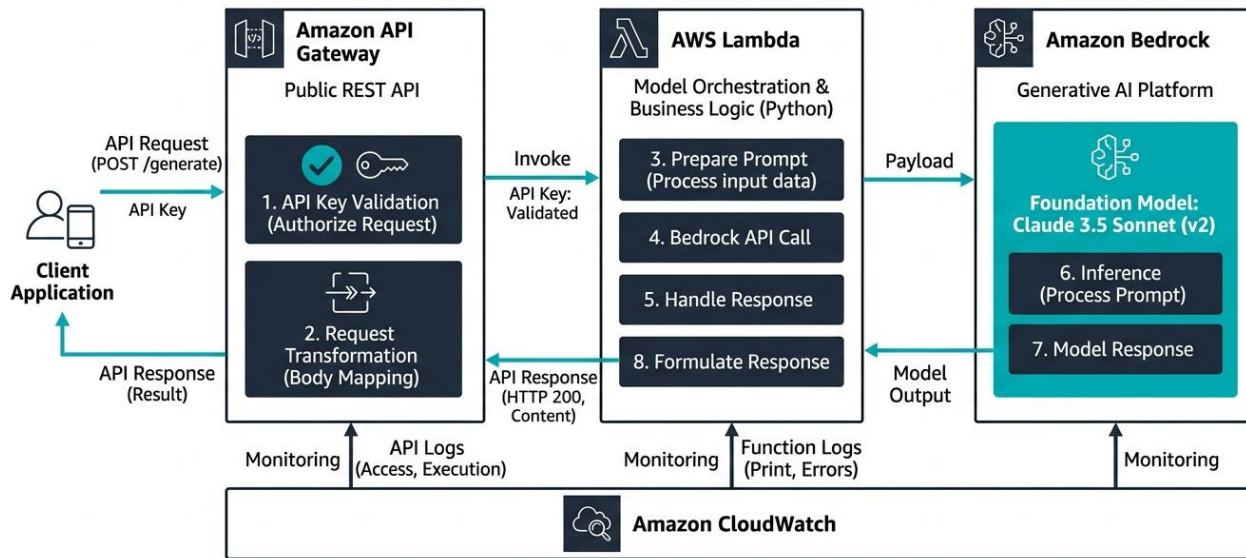
- Kubernetes-native deployments
- Multi-cloud or hybrid workloads
- Complex microservice architectures
- Hybrid Bedrock plus other ML models

Container Architecture Example

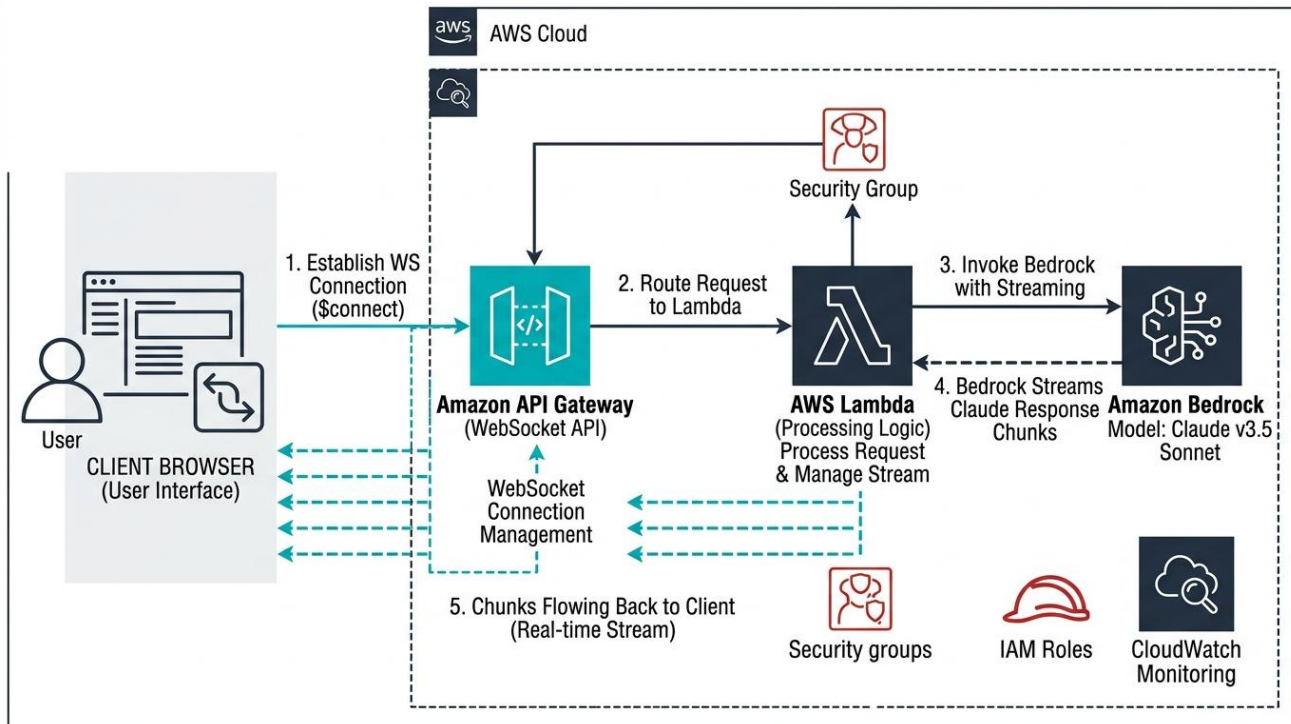


API Gateway Integration

AWS Architecture: API Gateway Integration with Amazon Bedrock (Claude 3.5 Sonnet)



Streaming Architecture with WebSocket



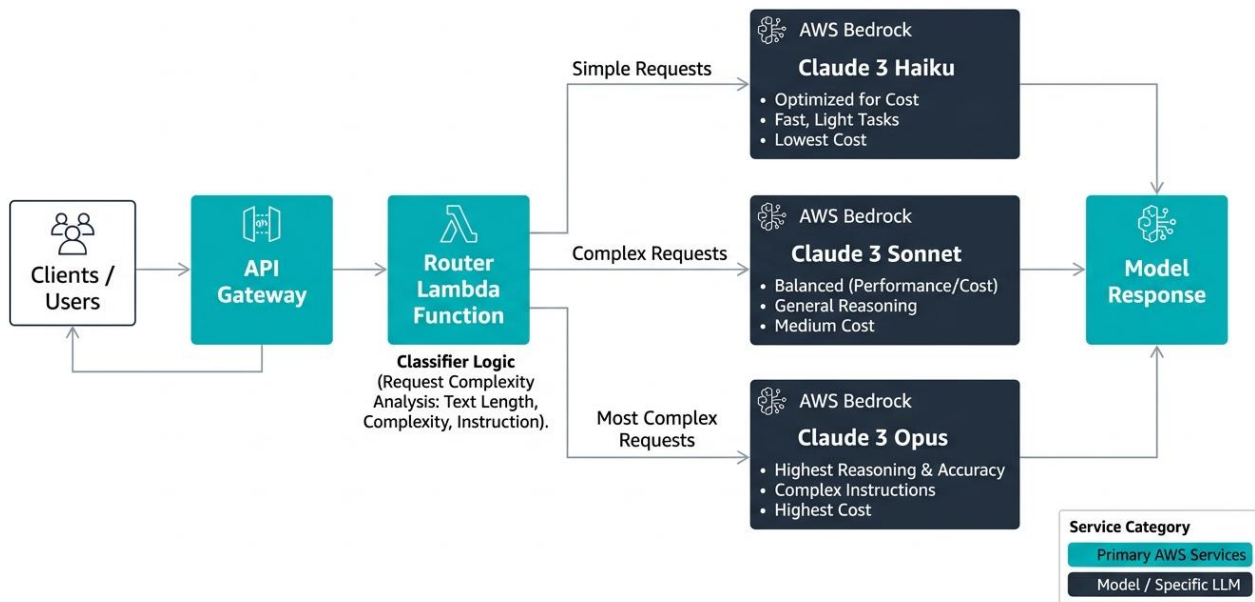


ROI Training

Cost Optimization

Dynamic Model Routing Architecture

Dynamic Model Routing for Cost Optimization on AWS (AWS Bedrock & Lambda)



Amazon Bedrock Service Tiers

Tier	Description	Use Case
On-Demand	Pay per token, standard availability	Development, variable workloads
Priority Tier	Guaranteed capacity, higher availability	Production with SLA requirements
Batch API	Lower cost, asynchronous processing	Bulk processing, non-urgent work

Service Tier Selection

1

Interactive applications: On-Demand or Priority Tier

2

Production with SLAs: Priority Tier

3

Bulk processing: Batch API (supports up to 300k output tokens)

4

Mixed workloads: Combine tiers strategically

Cost Optimization Strategies

1

Prompt optimization: Shorter prompts = fewer input tokens

2

Response length: Set appropriate max_tokens

3

Caching: Cache common query results

4

Quota management: Monitor and request increases proactively



ROI Training

Resilience

Resilience Implementation Patterns

1

Retry with exponential backoff + jitter

2

Circuit breakers for cascade prevention

3

Graceful degradation with fallbacks

4

Health checks and automated failover

Retry Logic with Exponential Backoff

```
import random
import time

def invoke_with_retry(request, max_retries=3):
    for attempt in range(max_retries):
        try:
            return bedrock_runtime.invoke_model(request)
        except ThrottlingException:
            delay = (2 ** attempt) + random.uniform(0, 1)
            time.sleep(delay)
    raise Exception("Max retries exceeded")
```


When to Add Multi-Region

- Add multi-region complexity only when reliability requirements justify it.
- Most workloads can run successfully in a single primary region with cross-region inference for failover.





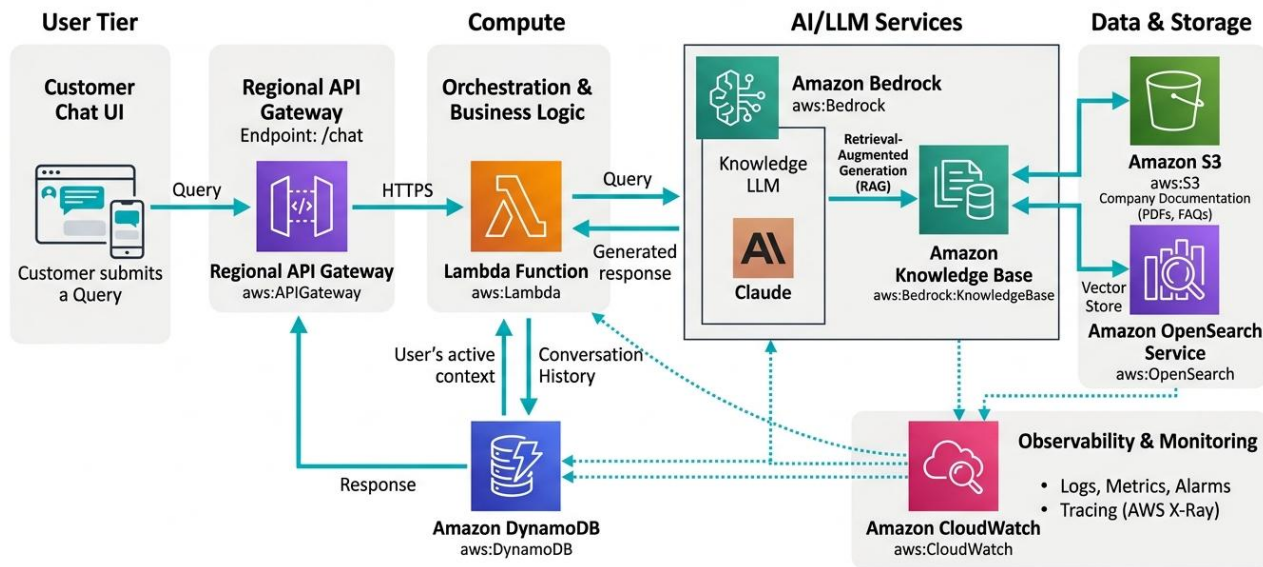
ROI Training

Demo

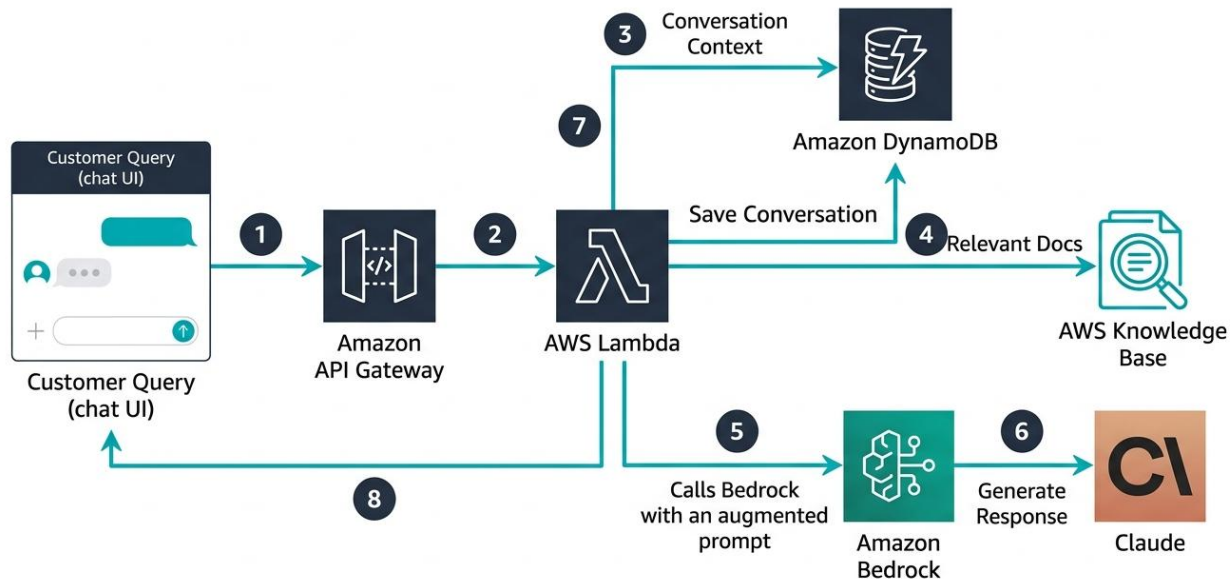
Complete Architecture: Customer Support Application

aws Customer Support Application leg Claude on Amazon Bedrock

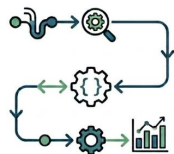
A Logical Request Flow



Request Flow

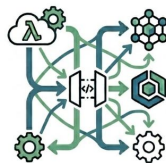


What We Covered



End-to-End Data Flows

Ingestion, retrieval, invocation, monitoring



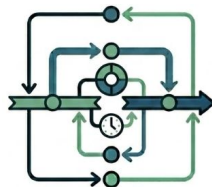
Integration Patterns

Lambda, ECS/EKS, and API Gateway



Optimize Costs

Model selection, service tier strategies, and cost optimization



Implement Resilience

Build robust systems with retry logic, circuit breakers, and failover strategies

Module 7 Is Next!

Module 1: Foundations: Generative AI and Claude Models on AWS

Module 2: Claude API Fundamentals and Prompt Engineering

Module 3: Data Preparation and Knowledge Bases for RAG

Module 4: Tool Use and Agentic Workflows with Claude

Module 5: Security, Governance, and Guardrails

Module 6: Practical Architecture and Deployment Patterns

► **Module 7: Claude Code: Building Bedrock Applications**

Module 8: Amazon Kiro and Enterprise Code Generation

